Ejercicio 05: motor recomendador reproducible

INF-8239 Ciencia de Datos II, Unidad 03 (U03.E05)

Estudiante	Roque Alexis Ventura Vallejo
Matrícula	100805873
Repositorio	github.com/RoqueVentura/UASD-Maestria-Ciencia-De-Datos, Ciencia de datos II/Unidad III/Ejercicio 05
Base	LAB08 y LAB09, proyecto INF8239_U03_Recomendadores_Proyecto_Base (uv, Python 3.12.13)

1. Evidencia de ejecución de los laboratorios

Ejecuté personalmente los dos laboratorios en mi equipo, desde VS Code. Las doce capturas (E1 a E12), cada una con lo que muestra y cómo la interpreto, están en un anexo aparte:

Anexo de evidencias: U03_E05_Evidencias_Roque_Ventura.pdf (abrir en GitHub)

2. Datos, licencia, auditoría y corte temporal

Usé MovieLens Latest Small de GroupLens Research (Harper y Konstan, 2015): valoraciones explícitas de 0.5 a 5 estrellas de 610 usuarios elegidos al azar entre quienes valoraron al menos 20 películas, sin datos demográficos. Es un dataset de desarrollo que puede cambiar, por eso registro el hash. Su licencia permite uso académico citando la fuente y prohíbe el uso comercial sin permiso. **No lo redistribuyo**: *data/raw/** está en .gitignore y se descarga con *scripts/download_data.py*.

Comprobación (reports/audit.json)	Resultado	Interpretación
Valoraciones, usuarios, películas	100,836 · 610 · 9,742	18 películas sin ninguna valoración: solo el contenido puede recomendarlas
Nulos, duplicados, ratings huérfanos	0 · 0 · 0	Contrato de datos válido; validate_movielens rechaza ratings fuera de [0.5, 5]
Densidad	1.70 %	La mayoría de pares no observados no significa rechazo
Periodo	1996-03-29 a 2018-09-24	Permite un corte temporal sin usar el futuro
Valoraciones por usuario	mín. 20, mediana 70.5, máx. 2,698	No hay usuarios fríos reales: el usuario nuevo se simula
Cola larga	3,446 películas con 1 valoración	Justifica suavizar la media en la popularidad

Corte temporal. Para cada usuario, su última valoración va a prueba y el resto a entrenamiento (100,226 y 610). Una prueba verifica que ninguna valoración de entrenamiento sea posterior a la de prueba del mismo usuario. Solo 587 usuarios son evaluables, porque en 23 casos la película de prueba no aparece en entrenamiento y ningún modelo colaborativo podría acertarla. Un corte aleatorio habría permitido aprender del futuro.

3. Popularidad, contenido y factorización

Popularidad suavizada. Combina la media de cada película con la media global, ponderadas por el número de valoraciones (mínimo: percentil 80 de los conteos). Así, una película con una sola valoración de 5 no domina. Top-3: Shawshank Redemption, The (1994) (4.395), Godfather, The (1972) (4.243), Fight Club (1999) (4.233). Sirve como referencia mínima y como fallback, pero da la misma lista a todas las personas.

Contenido. Representé los géneros con TF-IDF y comparé películas con similitud coseno. Probé tres consultas de géneros distintos (*reports/content_queries.csv*):

Consulta	Géneros	Top-10 con score 1.0	Películas del catálogo con géneros idénticos
Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy	10 de 10	12

Consulta	Géneros	Top-10 con score 1.0	Películas del catálogo con géneros idénticos
Godfather, The (1972)	Crime|Drama	10 de 10	133
Alien (1979)	Horror|Sci-Fi	10 de 10	52

La similitud significa «comparte los géneros registrados», no «le gustará». En las tres consultas todo el Top-10 empata en 1.0, así que el orden es arbitrario entre 12, 133 o 52 candidatas equivalentes. Por ejemplo, El Padrino recomienda *United 93* solo porque ambas están etiquetadas Crime|Drama. Esto es sobre-especialización: nunca sale de la misma combinación de géneros.

Factorización. Siguiendo a Koren, Bell y Volinsky (2009), aproximé cada rating como media global más el producto de vectores latentes del usuario y la película, entrenados con SGD (12 épocas, tasa 0.01, regularización 0.05). Con tres semillas, el RMSE medio fue 1.033 (10 factores), 1.026 (20) y 1.018 (40). La factorización mejora a la media global (1.116) y a la media por película (1.050). Un RMSE menor indica mejores ratings, no necesariamente un mejor Top-10.

4. Ranking, cobertura y usuarios fríos

Tabla principal. HitRate@10 es la proporción de los 587 usuarios cuya película retenida aparece en su Top-10. Precision@10 equivale a HitRate/10 porque hay un solo ítem relevante por usuario. Cobertura es la proporción del catálogo de entrenamiento (9,701 películas) que aparece en alguna lista. Para los modelos colaborativos doy la media ± desviación de las semillas 42, 123 y 2026.

Método	RMSE	HitRate@10	Aciertos (de 587)	Precision @10	Cobertura	Géneros por lista
Popularidad (no personalizada)	–	0.0426	25	0.0043	1.2 %	11.6
Contenido puro (perfil de géneros)	–	0.0017	1	0.0002	7.8 %	6.0
Colaborativo, 40 factores (alpha 1)	1.018	0.0335 ± 0.0120	20	0.0034	9.9 %	11.1
Híbrido LAB09 base (20 f., alpha 0.75)	1.026	0.0318 ± 0.0049	19	0.0032	7.4 %	10.7
Híbrido elegido (40 f., alpha 0.5)	1.018	0.0386 ± 0.0064	23	0.0039	10.8 %	9.3

La popularidad tiene el mayor HitRate (0.0426), pero recomienda los mismos 10 títulos a todos y cubre el 1.2 % del catálogo. Coincide con Cremonesi, Koren y Turrin (2010): en tareas Top-N, la popularidad es un baseline difícil de superar y el RMSE no predice el desempeño del ranking. El contenido puro casi no acierta, porque la última película vista rara vez es la más parecida en géneros. Las diferencias entre modelos colaborativos son de pocos usuarios: en las 45 corridas, los aciertos van de 13 a 29 de 587, por lo que no las trato como significativas.

Tres perfiles con la configuración elegida (*reports/profiles.csv*):

Perfil	Valoraciones	Primeros títulos recomendados	Comprobación
Historial amplio (usuario 414)	2,697	12 Angry Men (1957), Life Is Beautiful (La Vita è bella) (1997), Philadelphia Story, The (1940)	0 vistas en la lista; Jaccard entre semillas 0.54; comparte 4 títulos con alpha 1 y 4 con alpha 0
Historial pequeño (usuario 53)	19	First Knight (1995), Forrest Gump (1994), Curly Sue (1991)	0 vistas; Jaccard entre semillas 0.02: la lista cambia casi por completo; 7 de 10 coinciden con alpha 1
Usuario nuevo (611)	0	Forrest Gump (1994), Shawshank Redemption, The (1994), Pulp Fiction (1994)	El modelo devuelve 0 filas; el fallback da 10 títulos, marcados «no personalizada»

El usuario con historial amplio recibe dramas y comedias, coherentes con los géneros más frecuentes en su historial (Drama y Comedy). Con 19 valoraciones, la señal colaborativa es inestable: depende de la semilla más que del usuario. El usuario nuevo no tiene vector latente ni perfil, así que recibe la popularidad declarada como no personalizada.

5. Híbrido y decisión Pareto

El híbrido reordena las 100 películas mejor puntuadas por la factorización con *alpha · colaborativo + (1 – alpha) · contenido*, ambas señales normalizadas. Mantuve fijos el corte, las 12 épocas y las semillas. Cambié alpha (0 a 1) y los factores (10, 20, 40).

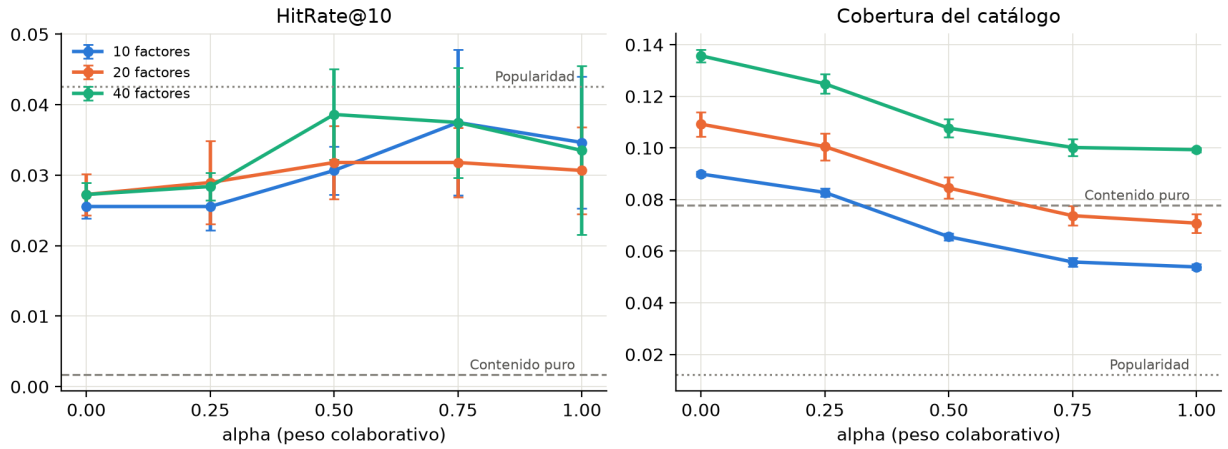


Figura 1. HitRate@10 y cobertura según alpha (media ± desviación de tres semillas). Las líneas grises son los baselines.

Factores	alpha	HitRate@10	Cobertura	Géneros	Tiempo medio (s)	Tamaño (MB)	¿Dominada?
10	0.00	0.0256 ± 0.0017	9.0 %	7.5	31.5	0.82	**No**
10	0.25	0.0256 ± 0.0034	8.3 %	8.1	31.5	0.82	Sí
10	0.50	0.0307 ± 0.0034	6.6 %	9.7	31.5	0.82	**No**
10	0.75	0.0375 ± 0.0104	5.6 %	10.8	31.5	0.82	**No**
10	1.00	0.0346 ± 0.0094	5.4 %	11.1	31.5	0.82	Sí
20	0.00	0.0273 ± 0.0030	10.9 %	7.3	24.1	1.65	**No**
20	0.25	0.0290 ± 0.0059	10.1 %	7.9	24.1	1.65	**No**
20	0.50	0.0318 ± 0.0052	8.4 %	9.6	24.1	1.65	**No**
20	0.75	0.0318 ± 0.0049	7.4 %	10.7	24.1	1.65	Sí
20	1.00	0.0307 ± 0.0061	7.1 %	11.1	24.1	1.65	Sí
40	0.00	0.0273 ± 0.0017	13.6 %	7.2	40.7	3.30	**No**
40	0.25	0.0284 ± 0.0020	12.5 %	7.6	40.7	3.30	**No**
40	0.50	0.0386 ± 0.0064	10.8 %	9.3	40.7	3.30	**No**
40	0.75	0.0375 ± 0.0078	10.0 %	10.6	40.7	3.30	Sí
40	1.00	0.0335 ± 0.0120	9.9 %	11.1	40.7	3.30	Sí

Utilidad: HitRate@10 y cobertura (más es mejor). Costo: tiempo de entrenamiento y tamaño de los factores (menos es mejor). Una configuración está dominada si otra es igual o mejor en todo y estrictamente mejor en algo.

Qué señal domina. Al subir alpha domina la señal colaborativa: la cobertura baja y las listas mezclan más géneros (de ~7 a ~11). Con alpha bajo domina el contenido: las listas se concentran en los géneros del usuario (burbuja de filtro), aunque recorren más catálogo. La configuración base del LAB09 (20 factores, alpha 0.75) está dominada por la misma con alpha 0.5: igual HitRate y más cobertura.

Costo. El tamaño se duplica con los factores (0.82, 1.65 y 3.30 MB). El tiempo varió entre 22 y 55 s sin relación con los factores, porque lo domina el bucle SGD en Python sobre 100,226 valoraciones y la carga del equipo (Intel Core i7-1165G7, 15.7 GB de RAM, sin GPU, Windows 11). Lo mismo se ve en E11 y E12: la misma configuración tardó 16.5 y 40.2 s.

Decisión. Elijo **40 factores con alpha 0.5**. No está dominada y tiene el mayor HitRate medio de los híbridos (0.0386) con 10.8 % de cobertura. Su alternativa más barata no dominada, 10 factores con alpha 0.75, logra un HitRate parecido (0.0375), pero con más variación entre semillas (±0.0104 frente a ±0.0064) y casi la mitad de cobertura (5.6 %). Siguiendo el criterio Green AI, no la elijo por ser la más grande: la mejora en HitRate no es concluyente. La justifican la cobertura y la estabilidad, y su costo extra son 2.5 MB de memoria sin un aumento de tiempo medible. La popularidad queda como fallback para usuarios nuevos, no como política principal, porque no personaliza y concentra la exposición.

6. Errores encontrados y cómo los resolví

Error	Causa	Solución
LAB09 fallaba con *TypeError: np.matrix is not supported*	En la línea 45 de lab09_hybrid.py, *.sum(axis=0)* sobre una matriz dispersa devuelve np.matrix, y scikit-learn 1.9.1 la rechaza en cosine_similarity	Convertí el perfil con *np.asarray* (E11 muestra el script modificado). Lo moví a *GenreProfile.profile* en src y lo cubre una prueba
El commit *feat: add popularity and content recommenders* quedó vacío (E9)	El primer commit usó *git add .* y ya incluyó todo el proyecto	Los cambios del ejercicio se registran en commits nuevos (sección 8)
Los reportes nunca se versionaban	.gitignore excluía *reports/**, así que *git add reports* no añadía nada	Cambié .gitignore para versionar reports/; data/raw/* sigue excluido para no redistribuir MovieLens
La misma configuración tardó 16.5 y 40.2 s (E11, E12)	Ruido de medición: carga del equipo, no el modelo	Promedié tres semillas y usé el tamaño como costo estable
No se podían comparar semillas ni configuraciones	lab09_hybrid.py no tenía *--seed* y sobrescribía hybrid_metrics.json en cada corrida	Añadí *--seed* y *e05_experiments.py*, que guarda las 45 corridas en experiments.csv

7. Límites

(1) Solo hay un ítem relevante por usuario y pocos aciertos (13 a 29 de 587), así que las métricas tienen mucha varianza. (2) El contenido usa solo géneros: 34 películas no tienen géneros y no usé etiquetas. (3) El híbrido solo reordena el pool colaborativo de 100 películas: una película nueva sin valoraciones solo la alcanza el contenido puro. (4) Hay sesgo de selección (se valora lo que se eligió ver) y no hay demografía, así que no puedo afirmar nada por grupos. (5) La evaluación es offline sobre un dataset de desarrollo: no mide satisfacción ni el efecto de retroalimentación de las recomendaciones. (6) El tiempo se midió en un portátil con una corrida por semilla.

8. Código, reproducibilidad, pruebas y Git

Ruta	Contenido
src/inf8239_u03/	data.py (descarga segura, carga y contrato), recommenders.py (popularidad, contenido, factorización vectorizada, perfil de géneros, híbrido, fallback, corte temporal), metrics.py (HitRate, Precision, cobertura, diversidad, Pareto)
scripts/	download_data, audit_data, lab08_content, lab09_hybrid (--seed), e05_experiments (baselines, 3 × 3 × 5 corridas, Pareto, figura), e05_profiles
notebooks/01_recomendador_movielens.ipynb	Notebook ejecutado: recalcula la auditoría, verifica el corte, repite las consultas y reentrena 40 factores (semilla 42). El RMSE, el HitRate y la cobertura coinciden con experiments.csv
reports/	audit.json, popular_top10.csv, content_queries.csv, baselines.csv, experiments.csv, pareto.csv, profiles.csv, profiles_checks.json, figures/
docs/, README.md	Dataset Card (fuente, hash, licencia, sesgos), System Card (señales, modelos, fallback, métricas, riesgos) e instrucciones
pyproject.toml, uv.lock, requirements.txt	Entorno fijado: uv como vía principal; requirements.txt exportado del lock para pip

```
uv sync
uv run pytest -q
uv run python scripts/download_data.py
uv run python scripts/audit_data.py
uv run python scripts/lab08_content.py
uv run python scripts/e05_experiments.py
uv run python scripts/e05_profiles.py --factors 40 --alpha 0.5
uv run jupyter nbconvert --to notebook --execute --inplace notebooks/01_recomendador_movielens.ipynb
```

Pruebas. 16 pruebas con pytest (16 passed en 3.1 s), sin necesidad de MovieLens. Las 7 originales cubren el contrato de datos, el contenido que excluye la consulta, la popularidad, el corte temporal, la factorización y las métricas. Añadí 9 que comprueban: que el Top-N excluye lo visto y coincide con *predict*, que un usuario desconocido no tiene señal colaborativa, que el fallback excluye lo visto, que la misma semilla da los mismos factores, que no se entrena con el futuro, que alpha 0 y 1 siguen cada señal, que el perfil prefiere los géneros valorados, Precision y diversidad, y que el Pareto marca la configuración dominada. Ruff no reporta errores.

Refactorización verificada. Después de mover el híbrido a src, *lab09_hybrid.py --factors 20 --epochs 12 --alpha 0.75* reproduce exactamente las métricas de E11: RMSE 1.0264183, HitRate@10 0.0374787 y cobertura 0.0775178.

9. Conclusión

Pregunta del cierre	Respuesta
Resultado de ratings	La factorización (RMSE 1.018 con 40 factores) mejora a la media global (1.116) y a la media por película (1.050).
Resultado de ranking	La popularidad acierta más (HitRate 0.043); el híbrido elegido logra 0.039 y el contenido puro 0.002. Menor RMSE no implicó mejor Top-10.
Qué representa la similitud	Compartir géneros registrados. En las tres consultas todo el Top-10 empata en 1.0.
Cobertura del catálogo	Popularidad 1.2 %, híbrido elegido 10.8 %. Sube con más factores y con más peso de contenido.
Usuario frío	El nuevo recibe un fallback de popularidad declarado como no personalizado. Con 19 valoraciones la lista cambia casi por completo entre semillas (Jaccard 0.02).
Configuración seleccionada	40 factores y alpha 0.5: no dominada, mejor cobertura y menor variación que la alternativa barata. La mejora en HitRate no es concluyente.
Costo computacional	3.3 MB de factores; tiempo de 22 a 55 s sin relación con los factores (ruido del equipo).
Riesgo principal y mitigación	Popularidad dominante y burbuja de géneros. Mitigación: alpha intermedio, medir cobertura y diversidad junto al HitRate, y limitar el fallback a usuarios nuevos.

Lo que más aprendí es que cada métrica cuenta una parte. El modelo con menor RMSE no ganó en ranking. El baseline más simple ganó en aciertos, pero a costa de recomendar lo mismo a todos. Y un alpha distinto cambia qué parte del catálogo ve cada persona. Sin las tres semillas habría tomado por mejora real una diferencia de dos o tres usuarios.

10. Uso de inteligencia artificial

Herramienta	Claude Code (modelo Claude Opus 5.5, de Anthropic), desde Visual Studio Code, en octubre de 2026, como apoyo durante el ejercicio.
Mi trabajo	Ejecuté LAB08 y LAB09 en mi equipo (E1–E12), identifiqué los errores que aparecieron y corregí el de np.matrix. Dirigí el armado del proyecto según el mandato y la rúbrica, ejecuté y revisé los experimentos, validé la decisión final y revisé el informe.
Consultas realizadas (resumidas)	Análisis de las guías LAB08 y LAB09 frente al proyecto base; consultas sobre las causas de los errores que encontré (np.matrix, commit vacío, tiempos distintos); apoyo para estructurar los experimentos de semillas, Pareto y perfiles; revisión del formato y la redacción del informe.
Apoyo recibido	Revisión del proyecto frente a la rúbrica; explicación de los errores; apoyo en la ampliación del código (híbrido en src, --seed, scripts de experimentos y perfiles, notebook y pruebas) y en la documentación (Dataset Card, System Card y README); revisión de la redacción.

Verificaciones	Las cifras salen de reports/, generado en mi equipo. El script refactorizado reproduce E11 exactamente. El notebook reentrena y coincide con experiments.csv. Las 16 pruebas pasan y ruff está limpio. Comprobé los tres DOI en doi.org y revisé la figura y los perfiles.
Correcciones realizadas	Durante la revisión se corrigió: (1) un bucle del script de experimentos que reconstruía el conjunto de vistos para cada candidato y era muy lento; (2) el tamaño del catálogo de entrenamiento en el System Card (son 9,701 películas, no 9,723); (3) el orden de imports y anotaciones señalado por ruff; (4) el .gitignore, para versionar la evidencia.
Responsabilidad	Revisé el contenido final, puedo explicar cada fragmento del código y asumo la responsabilidad de lo entregado.

11. Fuentes

Cremonesi, P., Koren, Y., & Turrin, R. (2010). Performance of recommender algorithms on top-N recommendation tasks. In *Proceedings of the Fourth ACM Conference on Recommender Systems* (pp. 39-46). https://doi.org/10.1145/1864708.1864721

Harper, F. M., & Konstan, J. A. (2015). The MovieLens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems, 5*(4), 1-19. https://doi.org/10.1145/2827872

Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. *Computer, 42*(8), 30-37. https://doi.org/10.1109/MC.2009.263